Data Definition Language (DDL)
Wiki: https://en.wikipedia.org/wiki/Data_definition_language

*** SCHEMA**
[SQL 2023] 11.1 <schema definition>

```
-- Create
--
CREATE SCHEMA myschema;
CREATE SCHEMA IF NOT EXISTS myschema;      -- Non-
standard
CREATE SCHEMA myschema
  AUTHORIZATION yui           -- owner

  --
  CREATE TABLE t1 ...         -- TABLE myschema.t1
  CREATE VIEW v1 ...          -- VIEW myschema.v1
  ...


-- Drop
DROP SCHEMA myschema
DROP SCHEMA IF EXSTS myschema;             -- Non-
standard
DROP SCHEMA myschema
  CASCADE
  RESTRICT
```

==========
======
== Columns + constraints ==

[SQL 2023] 11.4 <column definition>

Constraint
A constraint has a **name** used to reference a constraint
e.g.

```
CREATE TABLE t (x INT UNIQUE);

ALTER TABLE t DROP CONSTRAINT t_x_unique;
```

2 types of constraints
a) **column only i.e. single column** constraint:
Constraint that can only apply to single column.
MUST be defined with column
e.g. NOT NULL, DEFAULT

b) **table constraint**:
Constraint that can involve multiple columns.
e.g. UNIQUE, PRIMARY KEY, CHECK

*** Basic syntax**

e.g.

```
CREATE TABLE t (
  -- column definition
  id INT NOT NULL PRIMARY KEY,
  x VARCHAR NOT NULL,
  y VARCHAR NOT NULL,

  -- table constraint definition
  CONSTRAINT UNIQUE (x, y)
);

ALTER TABLE t
  -- column definition
  ADD COLUMN z INT DEFAULT 0,
 -- table constraint
  ADD CONSTRAINT CHECK(z != y)
```

```
-- column definition
id INT
id INT UNIQUE                                -- shorthand
constraint
                                             -- named
automatically e.g. t_id_unique
id INT CONSTRAINT UNIQUE                     -- explicit
CONSTRAINT
id INT CONSTRAINT idconstraint UNIQUE        -- explicit
name constraint
id INT UNIQUE NOT NULL                       -- multiple
constraints

name VARCHAR COLLATE "POSIX"                 -- collation

-- Types of column constraints
id INT
 -- 1) column constraints
 NOT NULL
 NULL                           -- [PostgreSQL] opposite
of NOT NULL
 DEFAULT 1


 -- 2) table constraints
 PRIMARY KEY,
 UNIQUE
 CHECK (id > 0)
 CHECK (id > x)           -- BAD. Cannot use other
columns in standard SQL. Use table constraint instead.
                          -- [PostgreSQL] OK
 REFERENCES t2 (uid)


-- Table constraints
```

```
CONSTRAINT c UNIQUE(x)        -- explicit name
CONSTRAINT UNIQUE(x)          -- named automatically
UNIQUE(x)                     -- same
FOREIGN KEY (id)
  REFERENCES t2 (uid)


- Types
CONSTRAINT UNIQUE(x)
CONSTRAINT UNIQUE(x, y)               -- multiple
CONSTRAINT PRIMARY KEY (x)
CONSTRAINT PRIMARY KEY (x, y)         -- multiple
CONSTRAINT CHECK (x > 0)
CONSTRAINT CHECK (x > y)              -- multiple
```

*** FOREIGN KEY**

[SQL 2023] 11.8 Referential constraint definition

```
-- Table constraint
FOREIGN KEY
  (x) REFERENCES t2 (a)           -- <table>.x = t2.a
  (x) REFERENCES t2               -- <table>.x =
t2.<primary key>
  (x, y)  REFERENCES t2 (a, b)    -- multiple

   -- Whether every row in this table must match a row
in the REFERENCE i.e. source
   MATCH SIMPLE                   -- NULL columns don't
require match [default]
   MATCH FULL                     -- every row must have
a match in the REFERENCE
   MATCH PARTIAL


   -- actions: handle changes in REFERENCE table
   --
   -- ON DELETE: when row on REFERENCE tries to DELETE
   ON DELETE NO ACTION            -- don't allow DELETE
[default]
   ON DELETE RESTRICT             -- similar
   ON DELETE CASCADE              -- also DELETE current
row
   ON DELETE SET NULL             -- set current table's
column to NULL
   ON DELETE SET DEFAULT          -- set to DEFAULT

   -- ON UPDATE: when the REFERENCE key tries to UPDATE
   ON UPDATE NO ACTION
   ...
```

*** GENERATED columns**

[SQL 2023] 4.17.12 Identity columns
[SQL 2023] 4.17.13 generated columns

GENERATED
An **auto-computed** column
2 types:

a) **IDENTITY** aka **auto-increment**:
Uses an underlying **SEQUENCE** to autogenerate values.
Default sequence begins from 1.
i.e. START WITH 1 INCREMENT BY 1 NO MINVALUE NO MAXVALUE

An IDENTITY column is either:

GENERATED BY DEFAULT	overridable by user value
GENERATED ALWAYS	not overridable EXCEPT when OVERRIDING SYSTEM VALUE

e.g. GENERATED ALWAYS AS IDENTITY

```
CREATE TABLE t (
  id INT GENERATED ALWAYS AS IDENTITY
);

-- next id1=1
INSERT INTO t DEFAULT VALUES;     -- row (id1: 1)

-- next id1=2
INSERT INTO t VALUES (10);        -- ERROR
INSERT INTO t                     -- OK. Override with
user value. row (id1: 10)
  OVERRIDING SYSTEM VALUE
  VALUES (10);
INSERT INTO t                     -- OK. Ignore user
value. row (id1: 2)
  OVERRIDING USER VALUE
  VALUES (100000);

-- next id1=3
INSERT INTO t                     -- row (id: 3)
  OVERRIDING SYSTEM VALUE
  VALUES (3);
INSERT INTO t DEFAULT VALUES;     -- row (id: 3)
                                  -- next sequence value
unaffected by override
```

e.g. GENERATED BY DEFAULT

```
CREATE TABLE t (
  id INT GENERATED BY DEFAULT AS IDENTITY
);

INSERT INTO t DEFAULT VALUES;     -- row (id1: 1)
INSERT INTO t VALUES (10);        -- OK. row (id: 10).
```

```
Can override.
INSERT INTO t                       -- OK. row (id1: 2)
  OVERRIDING USER VALUE
  VALUES (10);
```

b) **computed expression**: expression computed from other columns
Never overridable.

e.g.

```
CREATE TABLE t (
  x INT NOT NULL,
  y INT NOT NULL,
  sum INT GENERATED ALWAYS AS (x * y)
STORED           -- STORED required for PostgreSQL
);

INSERT INTO t VALUES (2, 3);               -- row (x: 2,
y: 3, sum: 5)
INSERT INTO t (x, y, sum)                  -- ERROR.
Computed column cannot be overridden
  OVERRIDNG SYSTEM VALUE
  VALUES (2, 3, 1);
```

```
-- CREATE TABLE
CREATE TABLE t (
  -- IDENTITY
  x INT GENERATED ALWAYS AS IDENTITY
  x INT GENERATED BY DEFAULT AS IDENTITY
  x INT GENERATED ... AS IDENTITY (            -- same,
with explicit sequence options
   START WITH 1
   INCREMENT BY 1
   NO MINVALUE
   NO MAXVALUE
  )

  -- computed
  sum INT GENERATED ALWAYS AS (x + y)
  sum INT GENERATED ALWAYS AS (x + y) STORED   --
[PostgreSQL] requires explicit STORED
)


-- INSERT with overrides
INSERT INTO t1 (id)
  OVERRIDING SYTEM VALUE          -- override generated
value
  OVERRIDING USER VALUE           -- ignore provided
value
                                  -- i.e. force use
```

```
generated value
  VALUES
   (1)                              -- override generated
value, if possible
   (DEFAULT)                        -- use generated value
```

==========

== Tables ==

*** CREATE TABLE**
Postgres: https://www.postgresql.org/docs/current/sql-createtable.html
[SQL 2023] Section 11.3 <table definition>

```
CREATE TABLE t1 (
  id INT UNIQUE NOT NULL           -- column definition
  ...

  CONSTRAINT PRIMARY KEY(x, y)     -- table constraint
definition
)
```

*** CREATE TABLE LIKE**
[SQL 2023] 11.3 <table definition>

CREATE TABLE LIKE
Copy table definition.
By default only copies column **data types** and **NOT NULL** constraint.
In particular, does NOT copy
- other constraints: e.g. UNIQUE, PRIMARY KEY, DEFAULT, CHECK, GENERATED, etc
- indexes

e.g.

```
-- case: LIKE only
CREATE TABLE t2 (LIKE t1);

 Column |       Type        | Collation | Nullable |
Default
========+===================+===========+==========+====
=====
 id     | integer           |           | not null |
 sum    | integer           |           |          |
 x      | integer           |           |          |
 y      | integer           |           |          |


-- case: LIKE with INCLUDING options
CREATE TABLE t1 (
```

```
  LIKE t2
   INCLUDING IDENTITY
   INCLUDING GENERATED
   INCLUDING CONSTRAINTS
   INCLUDING INDEXES
);


 Column |       Type        | Collation | Nullable |
Default
========+===================+===========+==========+====
=================================
 id     | integer           |           | not null |
generated always as identity
 sum    | integer           |           |          |
generated always as (x + y) stored
 x      | integer           |           |          |
 y      | integer           |           |          |
....

Indexes:
    "t2_pkey" PRIMARY KEY, btree (id)
    "t2_floor_excl" EXCLUDE USING btree (floor(f) WITH
=)
Check constraints:
    "b" CHECK (x > y)
    "c" CHECK (y > '-10'::integer)
```

```
CREATE TABLE t2 (
  LIKE t1,
  LIKE t2,                -- multiple

  x INT,                  -- can define columns
                          -- ERROR if exists already
  UNIQUE(id)              -- can define constraints


  -- INCLUDING: other things to copy
  LIKE t1                      -- copies data type and
NOT NULL only
   INCLUDING DEFAULTS         -- copy DEFAULT
   INCLUDING IDENTITY         -- copy GENERATED ... AS
IDENTITY
   INCLUDING GENERATED        -- copy GENERATED ... AS
<expression>
   EXCLUDING DEFAULTS         -- explicit exclude
   ...


   -- additional options
   -- e.g. Postgres
   INCLUDING CONSTRAINTS      -- copy CHECK constraint
```

```
   INCLUDING INDEXES          -- copy indexes + UNIQUE,
PRIMARY KEY, EXCLUDING constraints
   ...
)
```

*** TEMPORARY table**
[SQL 2023] 4.17.2.4 Temporary tables

TEMPORARY table
A table which exists only during **user session** i.e. while user is connected.
2 types of temporary tables:

LOCAL	visible only to owner
GLOBAL	visible to all users

NOTE: implementations typically only support LOCAL
Ref: https://www.postgresql.org/docs/current/sql-createtable.html#SQL-CREATETABLE-COMPATIBILITY

```
CREATE TEMPORARY TABLE t ...;
CREATE GLOBAL TEMPORARY TABLE t ...;
CREATE LOCAL TEMPORARY TABLE t ...;
CREATE TEMPORARY TABLE t ...
  ON COMMIT
   PRESERVE ROWS          -- Default. keep rows on commit
   DELETE ROWS            -- empty table on commit

   -- e.g. extra Postgres option
   DROP
```

*** CREATE TABLE OF type**
[SQL 2023]

TABLE OF type
A table defined using a **custom TYPE** as a row.

e.g.

```
CREATE TYPE user_type AS (
  firstname VARCHAR NOT NULL,
  lastname VARCHAR NOT NULL
);

CREATE TABLE user_table OF user_type (
  CONSTRAINT value_pk PRIMARY KEY(firstname, lastname)
);

INSERT INTO user_table VALUES ('abc', 'def');
```

```
CREATE TABLE mytable OF mytype (
  y INT,                          -- ERROR. Can't add
```

```
column

  -- constraints
  x UNIQUE,
  UNIQUE(x),                        -- same
  CONSTRAINT unique UNIQUE(x)       -- named
  UNIQUE(x, y)
)
```

*** CREATE TABLE AS**
[SQL 2023] 11.3 <table definition>

CREATE TABLE AS
Create table to load **data** i.e. query.
Only defines column **data types**.
i.e. no constraints.

e.g.

```
-- e.g. simple
CREATE table t3 AS SELECT 1 AS x, 'a' AS label;

 Column |  Type   | Collation | Nullable | Default
════════╪═════════╪═══════════╪══════════╪═════════
 x      | integer |           |          |
 label  | text    |           |          |


-- e.g. specify column name and type
CREATE table t3 AS SELECT CAST(1 AS SMALLINT), 'a' AS
label;

 Column |   Type   | Collation | Nullable | Default
════════╪══════════╪═══════════╪══════════╪═════════
 y      | smallint |           |          |
 label  | text     |           |          |


-- Table definition
CREATE TABLE t AS SELECT x, label FROM t1;     --
columns (x, label)
CREATE TABLE t (x, label) AS ...;              --
specify custom column names
CREATE TABLE t (x) AS ...;                     -- OK.
single column
CREATE TEMPORARY TABLE t AS ...;               --
TEMPORARY

-- Table content
CREATE TABLE t AS ... WITH DATA;               --
Default. Fills the table.
CREATE TABLE t AS ... WITH NO DATA;            -- Create
```

table only.

*** SELECT INTO**
[SQL 2023] 14.7 <select statement: single from>

```
-- SELECT INTO
--
-- Same as SELECT, but just
SELECT *
  INTO TABLE t2               -- create t2
  INTO TEMPORARY t2           -- temporary t2
  FROM t
```

*** ALTER TABLE**
[SQL 2023] 11.10 <alter table>

```
-- ALTER, RENAME
ALTER TABLE t1 RENAME TO t2;



ALTER TABLE t1 ADD v INT ...;
ALTER TABLE t1 ADD IF NOT EXISTS ...;

ALTER TABLE t1 RENAME CONSTRAINT x TO y;


-- columns
--
-- basic
ALTER TABLE t1 ADD COLUMN v3 INT;

-- multiple [non-Standard PostgreSQL]
--
ALTER TABLE t1
  ADD COLUMN v3 INT,
  ALTER COLUMN v4 INT,
  --;

-- column
ALTER TABLE t1
  -- ADD
  ADD COLUMN v3 INT       -- add column
  ADD v3 INT              -- same

  -- DROP
  DROP v              -- delete column
  DROP IF NOT EXSTS

  -- RENAME
  RENAME COLUMN x TO y
```

```
  -- ALTER
  ALTER v SET DATA TYPE bigint             -- change
type if castable
  ALTER v TYPE bigint                      -- same
  ALTER v SET DATA TYPE bigint             --
[PostgreSQL] custom cast
   USING CASE v
     WHEN 'a' THEN 1
     ELSE 2 END

  ALTER v ADD GENERATED AS IDENTITY        -- IDENTITY
  ALTER v DROP IDENTITY
  ALTER v DROP IDENTITY IF EXISTS
  ALTER v SET GENERATED AS (...)


  -- columnconstraints
  ALTER v SET DEFAULT 1                    -- DEFAULT
  ALTER v DROP DEFAULT
  ALTER v SET NOT NULL                     -- NOT NULL
  ALTER v DROP NOT NULL


-- table constraints
ALTER TABLE t1
  DROP CONSTRAINT c
  ADD CONSTRAINT c UNIQUE(v)
  DROP CONSTRAINT c



-- DROP, TRUNCATE
TRUNCATE: delete data
DROP TABLE t1             -- delete table
TRUNCATE TABLE t1         -- delete data
```

==========
======

== Sequence ==

[SQL 2023] 11.72 <sequence generator definition>

Sequence
Sequence of **integers**.
Used primarily to implement **autoincremented** columns.
i.e. **GENERATED AS IDENTITY** columns.

e.g. in PostgreSQL

```
-- Define
CREATE SEQUENCE myseq
  START 1
  INCREMENT BY 3
```

```
  MINVALUE -1
  MAXVALUE 9;

-- Use sequence
SELECT curval('myseq');       -- ERROR. No value
initially.

SELECT nextval('myseq');      -- value #1=1
SELECT curval('myseq');       -- 1
SELECT nextval('myseq');      -- value #2=4
SELECT curval('myseq');       -- 4
SELECT nextval('myseq');      -- value #3=7
SELECT curval('myseq');       -- 7
SELECT nextval('myseq');      -- ERROR. Max reached
SELECT curval('myseq');       -- 7 i.e. unchanged

-- Reset
SELECT setval('myseq', -5)    -- ERROR due to MINVALUE
SELECT setval('myseq', 2)
SELECT curval('myseq');       -- 2
SELECT nextval('myseq');      -- 5
```

Basic syntax

```
-- Create
--
CREATE SEQUENCE myseq;
CREATE SEQUENCE IF NOT EXISTS myseq;
CREATE SEQUENCE myseq
  -- Data type
  AS BIGINT
  AS INTEGER
  AS SMALLINT

  -- Sequence
  START WITH 1        -- default 0
  START 1
  INCREMENT BY 3      -- default 1
  NO MINVALUE         -- default
  MINVALUE 1
  NO MAXVALUE         -- default
  MAXVALUE 10

  -- cycle: loop when max is reached
  NO CYCLE            -- default
  CYCLE

  OWNED BY ...


CREATE TEMP SEQUENCE myseq;           -- valid during
session
CREATE UNLOGGED SEQUENCE myseq;       -- not logged
```

```
-- Drop
--
DROP SEQUENCE myseq;
DROP SEQUENCE myseq, myseq2;
DROP SEQUENCE IF EXISTS ...;


-- Alter
--
ALTER SEQUENCE myseq myseq
  START 1
  INCREMENT 2
  ...;
ALTER SEQUENCE myseq IF EXISTS myseq ...
ALTER SEQUENCE myseq LOGGED;
ALTER SEQUENCE myseq UNLOGGED;

-- common
ALTER SEQUENCE myseq RENAME TO myseq2;
ALTER SEQUENCE myseq OWNER TO yui;
ALTER SEQUENCE myseq SET SCHEMA temp;
```

[PostgreSQL] Sequence functions
Sequence manipulation: https://www.postgresql.org/docs/current/functions-sequence.html

```
currval('myseql')             -- current value. ERROR if
nextval has not been called.
nextval('myseq')              -- increment to next
lastval()                     -- value of last sequence
used

setvall('myseq', 1)
currval('myseq')              -- 1
setvall('myseq', 1, true)     -- same
setvall('myseq', 20, false)   -- value unchanged until
nextval()
currval('myseq')              -- still 1
nextval('myseq')              -- 20
```

==========

== View ==

[SQL 2023] 4.17.23 Derived tables
[SQL 2023] 11.32 <view definition>

*** Basic syntax**

```
CREATE VIEW myview                      -- basic
  AS SELECT 1 AS x, 'a' AS label;
```

```
CREATE VIEW myview (x, label)         -- explicit
columns
  AS ...

CREATE RECURSIVE VIEW myview          --
  AS ...<recursive query>...
```

```
-- e.g.
CREATE RECURSIVE VIEW nums_1_100 (n) AS
    VALUES (1)
UNION ALL
    SELECT n+1
    FROM nums_1_100       -- recursive: self-reference
nums_1_100
    WHERE n < 100;
```

```
-- other options
CREATE view myview
  AS SELECT ...

  WITH CHECKOPTION
  WITH LOCAL CHECKOPTION
  WITH CASCADE CHECKOPTION

-- [PostgreSQL]
CREATE TEMPORARY view myview AS ...
```

==========

== Data Manipulation Language (DML) ==

Data manipulation language (DML)
wiki: https://en.wikipedia.org/wiki/Data_Manipulation_Language
Basic write operations

*** INSERT**
[SQL 2023] 14.11 <insert statement>
Insert: https://en.wikipedia.org/wiki/Insert_(SQL)

```
-- VALUES
INSERT INTO t1 (name, value)
 VALUES ('a', 1)
INSERT INTO t1                    -- shorthand. Implicit
column names
 VALUES ('a', 1)
INSERT INTO t1 (name)             -- Specify subset of
columns
 VALUES ('a')                     -- Unspecified columns
should have an autogenerated value
                                  -- i.e. DEFAULT or
```

```
GENERATED
INSERT INTO t1                     -- explicit DEFAULT
 VALUES ('a', DEFAULT)
INSERT INTO                        -- multiple rows
 VALUES ('a', 1), ('b', 2), ...


INSERT INTO t1 () VALUES ()        -- ERROR. Can't insert
all defaults with VALUES
INSERT INTO t1 DEFAULT VALUES      -- OK. all defaults.
INSERT INTO t1 VALUES (DEFAULT)    -- same
INSERT INTO t1 VALUES              -- multiple
  (DEFAULT), (DEFAULT)


-- Subquery
INSERT INTO t1 (name, value)       -- explicit columns
  (SELECT n, v FROM t2 ....)
  (SELECT v, n FROM t2 ....)       -- ERROR. Should be in
the same order
INSERT INTO t1                     -- shorthand
  (SELECT n, v FROM t2 ....)
INSERT INTO t1 (name)              -- subset of columns
only
  (SELECT n FROM t2 ....)
```

*** UPDATE**

[SQL 2023] 14.14 <update statement: searched>
Update: https://en.wikipedia.org/wiki/Update_(SQL)

```
UPDATE t1
 SET name = 'b'
 SET name = DEFAULT                    -- use column
default
 SET name = 'b', value = 2;            --multiple values
 SET (name, value) = ('b', 2)          -- same
 SET (name, value) = ROW ('b', 2)

 -- subsquery
 SET name = (SELECT ...)               -- IMPORTANT: will
set NULL if SELECT is NULL
 SET name = (SELECT 'b'                -- ERROR. MUST
return exactly 1
             UNION SELECT 'a')

 -- WHERE
 WHERE name = 'a'
 WHERE name = (SELECT ...)             -- with subquery
in WHERE


 -- [Non standard] FROM
 -- Same as FROM in SELECT
```

```
 -- i.e. allow joining to another table
 -- e.g. Postgres: https://www.postgresql.org/docs/
current/sql-update.html
 SET name = t2.name
 FROM t2
 WHERE t1.id = t2.id
```

*** DELETE**
[SQL 2023] 14.9 <delete statement: searched>
DELETE: https://en.wikipedia.org/wiki/Delete_(SQL)

```
DELETE FROM t1;           -- delete all rows
DELETE FROM t1            -- specific
  WHERE value < 20
```

*** MERGE**
[SQL 2023] 14.12 <merge statement>
Wiki: https://en.wikipedia.org/wiki/Merge_(SQL)

MERGE
A generalize form of **manipulations** aka **INSERT** + **UPDATE** + **DELETE**.
In particular, allows manipulations in reference to other tables i.e. by **JOINs**

```
MERGE INTO t1
 -- USING ... ON
 -- join with a table expression
 --
 USING t2                      -- e.g. JOIN with table t2
 USING (SELECT ...) as t2      -- e.g. JOIN with subquery
 ON t1.id = t2.id              -- JOIN condition


  -- WHEN ...
  -- Manipulate based on the join
  --
  -- WHEN MATCHED (=inner join)
  -- i.e. exists in both t1 and t2
  WHEN MATCHED THEN
  WHEN MATCHED AND t1.id > 0 THEN          -- with
condition
    UPDATE SET x=t2.x, y=t2.y              -- same as
UPDATE t1 SET ...
    DELETE
    DO NOTHING


  -- WHEN NOT MATCHED (=right outer join)
  -- i.e. missing in t1 (=target table)
  WHEN NOT MATCHED THEN
  WHEN NOT MATCHED THEN AND t1.id > 10 THEN    -- with
```

```
condition
    INSERT (x, y) VALUES (1, 2), ...           -- same
as INSERT INTO t1 VALUES ...
    DO NOTHING


  -- [PostgreSQL only]
  --
  -- Also handle left outer join
  -- i.e. when missing in source
  --
  -- BY TARGET: right outer join
  -- i.e. missing in t1 (=target table)
  WHEN NOT MATCHED THEN
  WHEN NOT MATCHED BY TARGET THEN
    ...

  -- BY SOURCE: left outer join
  -- i.e. missing in t2 (=source table)
  WHEN NOT MATCHED BY SOURCE
    UPDATE SET ...
    DELETE
    DO NOTHING


-- multiple WHEN
-- conditions are checked in order
MERGE INTO t1
USING t2
ON t1.id = t2.id
  WHEN MATCHED AND t1.id > 90 THEN ...         --
checked first
  WHEN NOT MATCHED AND t1.id > 90 THEN ...     --
checked next
  WHEN MATCHED AND t1.id > 80 THEN ...
  ...
```

==========

== Function/Procedure ==

[SQL 2023] 11.60 <SQL-invoked function>

```
CREATE FUNCTION
  -- Arguments
  f()
  f(x INT)
  f(x INT, t INT)                -- multiple
  f(INT)                         -- unnamed
                                 -- i.e. only
positional reference $1, $2
```

```
--  parameter reference
CREATE FUNCTION f(x INT, y INT) RETURNS INT AS $$

  SELECT x + y;           -- same name
  SELECT $1 + $2;         -- same positional

$$ LANGUAGE SQL;
```

```
  -- composite type
  f(x mytype)                      -- composite type
```

```
-- e.g.
CREATE TYPE mytype AS (name VARCHAR, value INT);

-- a) function body
CREATE FUNCTION f(x mytype) RETURNS int AS $$

  SELECT x.value;
  SELECT $1.value;                          -- same with
positional
$$ LANGUAGE SQL;


-- b) function call
SELECT f(CAST(ROW('a', 1) AS mytype));     -- explicit
type
SELECT f(ROW('a', 1));                     -- implicit
CAST

-- e.g. function call with table row
-- CREATE FUNCTION g(mytable) ...

SELECT g(mytable.*) from mytable;      -- GOOD
SELECT g(mytable) from mytable;        -- OK
SELECT g(*) from mytable;              -- ERROR
```

```
  -- DEFAULT
  f(x int DEFAULT 0)                       --
                                           -- f(1)      x
= 1
                                           -- f()       x
= 0
  f(x int = 0)                             -- [Non
standard PostgreSQL] same as DEFAULT
  f(x int DEFAULT -1, y int DEFAULT -2)    -- multiple
  f(x int DEFAULT 0, y int)                -- ERROR.
  f(x mytype)

  -- positional parameters
  f(INT, BOOLEAN)                  -- $1, $2
```

```
CREATE FUNCTION f(int, int) RETURNS int AS $$
  SELECT $1 + $2
$$ LANGUAGE SQL
```

```
  -- IN/OUT
  f(x INT) ...
  f(IN x INT)    ...          -- same explicit


  -- OUT parameters
  f(OUT x INT) AS ...                      -- single
output
  f(OUT x INT, OUT b BOOLEAN)              -- multiple

  f(OUT x INT) RETURNS INT AS ...          -- OK but
RETURNS is not needed
  f(OUT x INT) RETURNS BOOLEAN AS ...      -- ERROR
```

```
-- call function: no need to set OUT parameters
> CREATE FUNCTION f(OUT x INT) AS 'select 1;' LANGUAGE
SQL;
> SELECT * FROM f();
 x
---
 1

-- call procedure: initialize OUT parameters
-- e.g. NULL
> CREATE PROCEDURE p(OUT x INT) AS 'select 1;' LANGUAGE
SQL;
> CALL p(NULL);
 x
---
 1
```

```
  -- RETURNS
  f() RETURNS INT AS ...                       -- scalar
  f() RETURNS mytype AS ...                    -- custom
type
  f() AS ...                                   -- no
return
  f() RETURNS VOID AS ...                      --
explicit no return
  f() RETURNS RECORD AS ...                    -- any
type (both scalar/composite)
  f() RETURNS TABLE(x INT, b BOOLEAN) AS ...   --
anonymous composite
                                               -- NOTE:
TABLE also allows multiple rows
```

                                                -- use
OUT parameters for one row

  -- NOTE: TABLE vs OUT parameter for anonymous output

```
-- TABLE vs OUT parameter
> CREATE FUNCTION f() RETURNS TABLE(x INT, b BOOLEAN) AS
$$
  VALUES (1, TRUE), (2, FALSE)
$$ LANGUAGE SQL;

> CREATE FUNCTION g(OUT x INT, OUT b BOOLEAN) RETURNS
RECORD AS $$
  VALUES (1, TRUE), (2, FALSE)
$$ LANGUAGE SQL;

> SELECT * FROM f();
 x | b
---+---
 1 | t
 2 | f

> SELECT * FROM g();
 x | b
---+---
 1 | t
```

  -- RETURNS TABLE: output table
  -- i.e. output multiple rows
  --
  f() RETURNS TABLE(x INT) AS ...   --
rows of scalars
  f() RETURNS TABLE(x INT, b BOOLEAN) AS ...   --
rows of anonymous columns

```
-- e.g.
> CREATE FUNCTION f() RETURNS TABLE(x INT) AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;

> SELECT * FROM f();
 x
---
 1
 2
```

  -- [Non-standard][PostgreSQL] with SETOF
  f() RETURNS SETOF INT AS ...   --
rows of scalars
  f() RETURNS SETOF mytype AS ...   --

```
rows of composites
  f() RETURNS SETOF RECORD AS ...                    --
rows of any types
  f(OUT x INT, OUT b BOOLEAN)                        --
rows of anonymous columns
    RETURNS SETOF RECORD AS ...
```

```
-- e.g.
> CREATE FUNCTION f() RETURNS SETOF INT AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;

> SELECT * FROM f();
 f
---
 1
 2
```

```
  -- GOTCHA: for some reason, SETOF RECORD does not work
for single composite OUT
  f(OUT mytype) RETURNS SETOF RECORD
AS ...           -- ERROR 'function result type must be
mytype because of OUT parameters'
  f(OUT mytype) RETURNS SETOF mytype
AS ...           -- OK but redundant
  f() RETURNS SETOF mytype
AS ...                          -- GOOD
  f(OUT mytype, OUT INT) RETURNS SETOF RECORD
AS ...  -- GOOD For some reason, it works if there is
another OUT.
```

*** RETURNS vs OUT parameters**

In general:

RETURNS	**single valued** output	
	e.g.	
	RETURNS INT	single scalar
	RETURNS mytype	single composite (with multiple fields)
	RETURNS TABLE(x INT, y INT)	single table (with multiple columns)
OUT parameter	**anonymous multi-valued** output Can also assign **names** to columns.	

e.g.

```
-- RETURNS
--
-- scalar
```

```
> CREATE FUNCTION f1() RETURNS INT AS 'SELECT 1 AS x'
LANGUAGE SQL;
> SELECT * FROM f1();
 f1
────
  1


-- composite type
> CREATE TYPE mypoint AS (x INT, y INT);
> CREATE FUNCTION f2() RETURNS mypoint AS 'SELECT
ROW(1,2)' LANGUAGE SQL;
> SELECT * FROM f2();
 x | y
───┼───
 1 | 2

-- OUT param
> CREATE FUNCTION g1(OUT x INT, OUT y INT) AS 'SELECT 1,
2' LANGUAGE SQL;
> SELECT * FROM g1();
 x | y
───┼───
 1 | 2


 f1
────
 1
```

RETURNS TABLE(...)
Returns a table.
Can still be understood as **single-valued** .
i.e. a **single table** instance.
e.g.

```
-- case: table with 1 column
> CREATE FUNCTION f() RETURNS TABLE(x INT) AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;

> SELECT * FROM f();
 x
───
 1
 2

-- case: table with multiple columns
> CREATE FUNCTION f() RETURNS TABLE(x INT, t BOOLEAN) AS
$$
  VALUES (1, T), (2, F);
$$ LANGUAGE SQL;
```

```
> SELECT * FROM f();
 x | t
---+---
 1 | t
 2 | f
```

==========

== Transaction controls ==

[SQL 2023] 4.43 SQL-transactions
[SQL 2023] 17.1 <start transaction statement>

Isolation level
Defines what **changes (INSERT/DELETE/UPDATE)** made by **other transactions** is visible in the current transaction.

	changes by another transaction		
	INSERT	**DELETE**	**UPDATE**
READ COMMITTED	yes	yes	no
READ UNCOMMITTED	yes	yes	yes
REPEATABLE READ	yes	no	no
SERIALIZABLE	no	no	no

A transaction fails if the isolation cannot be satisifed
e.g.

```
-- user1
> START TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- OK: initial values
> SELECT value FROM t;
 value
-------
 a
 a
             -- user2: change value

             > UPDATE TABLE t SET value='b';


-- OK but changes by user2 not visible
> SELECT value FROM t;
```

```
 value
_______
 a
 a

-- ERROR
> UPDATE TABLE t SET value='c';
ERROR:  could not serialize access due to concurrent
update
```

```
START TRANSACTION
  -- Access
  READ ONLY
  READ WRITE

  -- Isolation
  ISOLATION LEVEL READ UNCOMMITTED
  ISOLATION LEVEL READ COMMITTED
  ISOLATION LEVEL REPEATABLE READ
  ISOLATION LEVEL SERIALIZABLE

  DIAGNOSTIC SIZE


COMMIT;
COMMIT WORK;                        -- same as COMMIT.
COMMIT AND CHAIN;                   -- COMMIT and start
new transaction.
COMMIT AND NO CHAIN;

SAVE;
COMMIT;
ROLLBACK;
```

```
 ============================================================
                              ======
                            == OLD ===
```

```
-- case: with OUT
> CREATE FUNCTION f2(OUT x INT) AS $$
  SELECT 1;
$$ LANGUAGE SQL;

> SELECT f2();

 f2
```

```
──
 1

> SELECT * FROM f2();          -- same named

 x
──
 1
```

Thus, there are 2 ways to output **multiple values**
1) **RETURNS** a **composite type**

e.g.

```
> CREATE TYPE mytype AS (name VARCHAR, value INT);

> CREATE FUNCTION f() RETURNS mytype AS $$
  SELECT 'a', 1;
$$ LANGUAGE SQL;

> SELECT f();

  f
───────
 (a,1)

> SELECT * FROM f();

 name | value
──────┼───────
 a    |     1
```

2) multiple **OUT** parameters
i.e. same as returning an **anonymous** composite type
e.g.

```
> CREATE FUNCTION f(OUT x INT, OUT y BOOLEAN) AS $$
  SELECT 1, TRUE
$$ LANGUAGE SQL

-- as single column
> SELECT f();
   f
───────
 (1,t)

-- as separate columns
> SELECT * FROM f();
```

```
 x | y
---+---
 1 | t
```

```
CREATE FUNCTION f()
  RETURNS INT AS ...                        -- scalar
  RETURNS mytype AS ...                     -- composite
  RETURNS TABLE(x INT, b BOOLEAN) AS ...    -- anonymous
composite

CREATE FUNCTION f() RETURNS INT AS $$
  SELECT 1 AS x;
$$ LANGUAGE SQL;

  -- multiple rows
  RETURNS SETOF INT AS ...
  RETURNS SETOF mytype As ...
  RETURNS TABLE(x INT, b BOOLEAN) AS ...

CREATE PROCEDURE
  p(OUT x INT) AS ...
  p(OUT INT) AS ...
  p(OUT x mytype) AS ...
  p(OUT x INT, OUT b BOOLEAN) AS ...

  -- multiple rows
  p(OUT INT) AS SETOF INT
SELECT f();

SELECT * FROM f();


  (x INT, OUT y INT) AS ...        -- same using OUT


  -- case: single value
CREATE FUNCTION f() RETURNS INT AS ...          -- GOOD
CREATE FUNCTION f(OUT INT) AS ...               -- OK
Same, but prefer RETURNS
CREATE FUNCTION f(OUT x INT) AS ...             -- same
named


-- case: composite type
CREATE FUNCTION f() RETURNS mytype AS ...       -- GOOD
CREATE FUNCTION f(OUT mytype) AS ...            -- OK
Same, but prefer returns
CREATE FUNCTION f(OUT x mytype) AS ...          -- same
named
```

```
-- case: multiple values
-- i.e. anonymous type
CREATE FUNCTION f(OUT x INT, OUT y INT)
AS ...            -- GOOD
CREATE FUNCTION f(OUT INT, OUT INT)
AS ...                -- OK named
CREATE FUNCTION f() RETURNS TABLE(x INT, y INT)
AS ...   -- OK but returns is not necessary


-- Syntax
CREATE FUNCTION f
  -- RETURNS
  () RETURNS INT AS ...
  (OUT INT) AS ...             -- OK. Same, but prefer
RETURNS
  (OUT x INT) AS ...           -- same but unnamed


  -- OUT
  (x INT, OUT double INT, OUT triple INT) AS ...
  (x INT, OUT INT, OUT INT) AS ...
             -- OK, but unnamed
```

```
-- case: named
> CREATE FUNCTION f(OUT x INT, OUT y BOOLEAN) AS $$
  SELECT 1, TRUE
$$ LANGUAGE SQL

> SELECT * FROM f();

 x | y
---+---
 1 | t

-- case unnamed
CREATE FUNCTION f(OUT INT, OUT BOOLEAN) AS $$
  SELECT 1, TRUE
$$ LANGUAGE SQL

> SELECT * FROM f();

 column1 | column2
---------+---------
       1 | t
```

```
  -- OUT + RETURNS
  (x INT, OUT double INT, OUT triple INT) RETURNS RECORD
AS ...       -- [Non-standard PostgreSQL] OK, but
unnecessary
  (x INT, OUT double INT, OUT triple INT) RETURNS INT
AS ...       -- ERROR OUT does not match RETURNS
```

```
  (x INT) RETURNS TABLE(double INT, triple INT) AS ...
            -- similar, but may output multiple rows



  -- case: single OUT
  (x INT, OUT double INT) AS ...                    --
OK, but prefer RETURNS
  (x INT) RETURNS INT AS ...                        --
same i.e. unnamed
```

*** RETURNS TABLE i.e. RETURNS multiple rows**

```
CREATE FUNCTION
  f() RETURNS TABLE(x INT, y BOOLEAN) AS
  f() RETURNS TABLE(INT, BOOLEAN) AS       -- ERROR
columns must have names
  f() RETURNS TABLE(x INT) AS              -- single
value



  -- [Non-standard PostgreSQL] SETOF
  f() RETURNS SETOF INT                             --
same
  f(OUT x
  f(OUT x INT, OUT y BOOLEAN) RETURNS SETOF RECORD  --
anonymous type
  f() RETURNS TABLE(x INT, y BOOLEAN) AS            --
same with TABLE
  f() RETURNS SETOF RECORD                          --
BAD Compiles but fails to call
```

```
-- e.g.
CREATE FUNCTION f() RETURNS TABLE(x INT, t BOOLEAN)
AS ...
CREATE FUNCTION f() RETURNS TABLE(x INT, t BOOLEAN)
AS ...
```

```
CREATE FUNCTION f() RETURNS SETOF INT AS $$

CREATE FUNCTION f(OUT x INT) RETURNS SETOF INT AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;
```

```
CREATE FUNCTION f() RETURNS INT AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;

CREATE FUNCTION f() RETURNS TABLE(x INT) AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;

CREATE FUNCTION f() RETURNS SETOF INT AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;

CREATE FUNCTION f(OUT x INT) RETURNS SETOF INT AS $$
  VALUES (1), (2);
$$ LANGUAGE SQL;

CREATE FUNCTION f(OUT x INT, OUT t BOOLEAN) RETURNS
SETOF RECORD AS $$
  VALUES (1, TRUE), (2, FALSE);
$$ LANGUAGE SQL;


 x
───
 1
 2


CREATE FUNCTION f(OUT x INT, OUT y BOOLEAN) RETURNS
SETOF RECORD AS $$
  VALUES (1, TRUE), (2, FALSE);
$$ LANGUAGE SQL;

 x | y
───┼───
 1 | t
 2 | f

CREATE FUNCTION f(IN x int, OUT y VARCHAR, OUT z
INT) ...
```